

PC FORGE

Ghidul bonusurilor PC Forge

Etapele 1–8

Ce pagină deschizi • ce acțiune faci • ce fișier arăți • ce explicație dai

Etapele 1–8 • audit complet, explicații în context și demonstrații

Cum folosești acest ghid

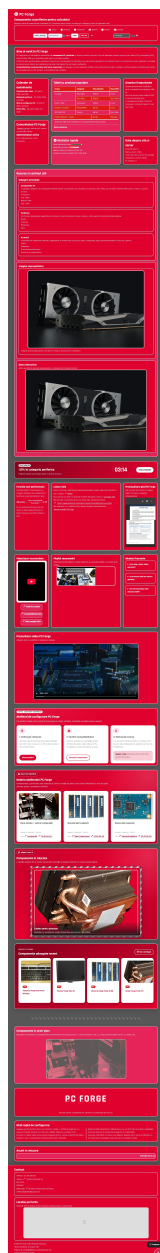
SCOP: Ghidul verifică bonusurile etapelor 1–8 și îți oferă explicația în ordinea în care o poți spune la prezentare. Funcțiile nu sunt puse separat: apar direct lângă cerința pe care o rezolvă.

METODA: Pentru fiecare bonus pornești de la ce se vede, continui cu datele și funcția, apoi închei cu motivul alegerii. Nu memora codul caracter cu caracter; memorează intrarea, transformarea și rezultatul.

STATUS: Etapele 1–3 erau complete. Etapa 4 a fost completată cu validarea integrală erori.json. În etapa 6 au fost completate curățarea backupurilor și imaginile corecte ale produsului. În etapa 8 au fost consolidate preferința. Rămâi conectat, drepturile active și e-mailul de stoc redus.

ETAPA 1 — HTML, resurse integrate și media

REZULTATUL AUDITULUI: Sunt documentate 5 bonusuri. Fiecare a fost urmărit de la cerință până la fișierul și demonstrația sa.



Reper vizual pentru Etapa 1

Bonus 1.1 — Formula MathML

CE CERE: O formulă relevantă site-ului, scrisă semantic, nu ca imagine.

CUM ESTE REZOLVAT ȘI DE CE: În `index.ejs`, elementele MathML `mfrac`, `mrow`, `mtext` și `mn` descriu raportul dintre scor și preț. Browserul construiește fracția, iar cititoarele de ecran păstrează sensul matematic.

CUM ÎL DEMONSTREZI: Pe pagina principală arată formula și explic de ce MathML este mai bun decât o captură.

FIȘIERELE PE CARE LE ARĂȚI: `views/pagini/index.ejs`

BUCATA-CHEIE: `<math><mfrac>...</mfrac></math>`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `views/pagini/index.ejs`. Ideea principală este: În `index.ejs`, elementele MathML `mfrac`, `mrow`, `mtext` și `mn` descriu raportul dintre scor și preț. Browserul construiește fracția, iar cititoarele de ecran păstrează sensul matematic. În browser demonstrez astfel: Pe pagina principală arăt formula și explic de ce MathML este mai bun decât o captură.

Bonus 1.2 — Document PDF integrat

CE CERE: Un PDF afișat direct într-o pagină.

CUM ESTE REZOLVAT ȘI DE CE: Elementul `embed` primește calea documentului și tipul `application/pdf`. Browserul folosește propriul vizualizator fără a copia conținutul PDF în HTML.

CUM ÎL DEMONSTREZI: Deschid secțiunea cu ghidul PDF și derulez documentul în pagină.

FIȘIERELE PE CARE LE ARĂȚI: `views/pagini/index.ejs`

BUCATA-CHEIE: `<embed src="...pdf" type="application/pdf">`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `views/pagini/index.ejs`. Ideea principală este: Elementul `embed` primește calea documentului și tipul `application/pdf`. Browserul folosește propriul vizualizator fără a copia conținutul PDF în HTML. În browser demonstrez astfel: Deschid secțiunea cu ghidul PDF și derulez documentul în pagină.

Bonus 1.3 — Hartă de imagine

CE CERE: Zone diferite ale aceleiași imagini trebuie să fie linkuri.

CUM ESTE REZOLVAT ȘI DE CE: Atributul `usemap` leagă imaginea de map; elementele `area` definesc dreptunghi, cerc și poligon prin coordonate. Fiecare zonă are `href` și `alt` propriu.

CUM ÎL DEMONSTREZI: Trec cu mouse-ul și apăs pe două zone distincte.

FIȘIERELE PE CARE LE ARĂȚI: `views/pagini/index.ejs`

BUCATA-CHEIE: `<area shape="rect|circle|poly" coords="...">`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `views/pagini/index.ejs`. Ideea principală este: Atributul `usemap` leagă imaginea de map; elementele `area` definesc dreptunghi, cerc și poligon prin coordonate. Fiecare zonă are `href` și `alt` propriu. În browser demonstrez astfel: Trec cu mouse-ul și apăs pe două zone distincte.

Bonus 1.4 — Google Maps

CE CERE: O hartă Google integrată în site.

CUM ESTE REZOLVAT ȘI DE CE: `Iframe`-ul din footer încarcă resursa Maps într-o zonă izolată. Este pus în fragment pentru a apărea consecvent pe pagini.

CUM ÎL DEMONSTREZI: Cobor în footer și arăt că harta poate fi deplasată.

FIȘIERELE PE CARE LE ARĂȚI: `views/fragmente/footer.ejs`

BUCATA-CHEIE: `<iframe src="https://www.google.com/maps/embed...">`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `views/fragmente/footer.ejs`. Ideea principală este: `Iframe`-ul din footer încarcă resursa Maps într-o zonă izolată. Este pus în fragment pentru a apărea consecvent pe pagini. În browser demonstrez astfel: Cobor în footer și arăt că harta poate fi deplasată.

Bonus 1.5 — YouTube cu autoplay și buclă

CE CERE: Clipul trebuie redat automat, fără sunet, repetat și cu controale.

CUM ESTE REZOLVAT ȘI DE CE: Parametrii `playlist`, `autoplay`, `mute`, `loop` și `controls` sunt transmiși în URL. `Mute` este important deoarece browserele blochează de regulă `autoplay`-ul cu sunet.

CUM ÎL DEMONSTREZI: Reîncarc pagina și indic pornirea automată și controalele.

FIȘIERELE PE CARE LE ARĂȚI: `views/pagini/index.ejs`

BUCATA-CHEIE: `?autoplay=1&mute=1&loop=1&playlist=ID&controls=1`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `views/pagini/index.ejs`. Ideea principală este: Parametrii `playlist`, `autoplay`, `mute`, `loop` și `controls` sunt transmiși în URL. `Mute` este important deoarece browserele blochează de regulă `autoplay`-ul cu sunet. În browser demonstrez astfel: Reîncarc pagina și indic pornirea automată și controalele.

ETAPA 2 — Reset CSS și stilizarea formulei

REZULTATUL AUDITULUI: Sunt documentate 2 bonusuri. Fiecare a fost urmărit de la cerință până la fișierul și demonstrația sa.

Bonus 2.1 — Reset CSS separat

CE CERE: Resetarea stilurilor implicite trebuie să fie într-un fișier separat.

CUM ESTE REZOLVAT ȘI DE CE: `reset.css` normalizează marginile, `box-sizing` și comportamentul elementelor înaintea temei PC Forge. Ordinea din head îl încarcă înaintea stilurilor proiectului.

CUM ÎL DEMONSTREZI: Compar pagina cu CSS activ și indic ordinea linkurilor.

FIȘIERELE PE CARE LE ARĂȚI: `resurse/css/reset.css`; `views/fragmente/head.ejs`

BUCATA-CHEIE: `*, *::before, *::after { box-sizing: border-box; }`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `resurse/css/reset.css`. Ideea principală este: `reset.css` normalizează marginile, `box-sizing` și comportamentul elementelor înaintea temei PC Forge. Ordinea din head îl încarcă înaintea stilurilor proiectului. În browser demonstrez astfel: Compar pagina cu CSS activ și indic ordinea linkurilor.

Bonus 2.2 — Stilizarea MathML

CE CERE: Formula din etapa 1 primește stiluri proprii.

CUM ESTE REZOLVAT ȘI DE CE: Selectorii MathML controlează culoarea, spațierea și evidențierea număratorului și numitorului, păstrând structura semantică.

CUM ÎL DEMONSTREZI: Inspectez formula și schimb temporar o proprietate în DevTools.

FIȘIERELE PE CARE LE ARĂȚI: resurse/css/stil.css

BUCATA-CHEIE: `math mfrac { ... }`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `resurse/css/stil.css`. Ideea principală este: Selectorii MathML controlează culoarea, spațierea și evidențierea număratorului și numitorului, păstrând structura semantică. În browser demonstrez astfel: Inspectez formula și schimb temporar o proprietate în DevTools.

ETAPA 3 — Meniu responsive și animații Sass

REZULTATUL AUDITULUI: Sunt documentate 3 bonusuri. Fiecare a fost urmărit de la cerință până la fișierul și demonstrația sa.

Bonus 3.1 — Hamburger din trei span-uri

CE CERE: Pictograma meniului trebuie construită din trei elemente, nu dintr-o imagine.

CUM ESTE REZOLVAT ȘI DE CE: Cele trei span-uri formează bare independente, ceea ce permite animarea și adaptarea lor prin CSS.

CUM ÎL DEMONSTREZI: Micșorez fereastra și arăt structura butonului.

FIȘIERELE PE CARE LE ARĂȚI: `views/fragmente/header.ejs`; `resurse/sass/meniu.scss`

BUCATA-CHEIE: `<span></span><span></span><span></span>`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `views/fragmente/header.ejs`. Ideea principală este: Cele trei span-uri formează bare independente, ceea ce permite animarea și adaptarea lor prin CSS. În browser demonstrez astfel: Micșorez fereastra și arăt structura butonului.

Bonus 3.2 — Animație în trei cadre

CE CERE: Barele își schimbă culoarea, transformarea și opacitatea.

CUM ESTE REZOLVAT ȘI DE CE: Keyframes are început, stare intermediară și final. Transform mută sau rotește fără recalcularea întregului layout, iar opacity produce dispariția graduală.

CUM ÎL DEMONSTREZI: Deschid meniul mobil și urmăresc mișcarea barelor.

FIȘIERELE PE CARE LE ARĂȚI: `resurse/sass/meniu-animatii.scss`

BUCATA-CHEIE: `@keyframes ... { 0% {...} 50% {...} 100% {...} }`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `resurse/sass/meniu-animatii.scss`. Ideea principală este: Keyframes are început, stare intermediară și final. Transform mută sau rotește fără recalcularea întregului layout, iar opacity produce dispariția graduală. În browser demonstrez astfel: Deschid meniul mobil și urmăresc mișcarea barelor.

Bonus 3.3 — Întârzieri generate cu Sass

CE CERE: Cele trei bare pornesc pe rând.

CUM ESTE REZOLVAT ȘI DE CE: Bucla `@for` generează câte un selector `nth-child` și calculează delay-ul din index: 0, 100 și 200 ms. CSS-ul repetitiv este produs la compilare.

CUM ÎL DEMONSTREZI: Arăt fișierul SCSS și apoi regulile compilate.

FIȘIERELE PE CARE LE ARĂȚI: `resurse/sass/meniu-animatii.scss`

BUCATA-CHEIE: `@for $i from 1 through 3 { &:nth-child(#{$i}) { ... } }`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `resurse/sass/meniu-animatii.scss`. Ideea principală este: Bucla `@for` generează câte un selector `nth-child` și calculează delay-ul din index: 0, 100 și 200 ms. CSS-ul repetitiv este produs la compilare. În browser demonstrez astfel: Arăt fișierul SCSS și apoi regulile compilate.

ETAPA 4 — Validarea fișierului de erori

REZULTATUL AUDITULUI: Sunt documentate 1 bonusuri. Fiecare a fost urmărit de la cerință până la fișierul și demonstrația sa.

Bonus 4.1 — Validarea completă erori.json

CE CERE: La pornire se verifică existența, JSON-ul, proprietățile, imaginile, identificatorii unici și câmpurile necunoscute.

CUM ESTE REZOLVAT ȘI DE CE: Funcția `valideazaDateErori` citește fișierul și acumulează toate problemele. `gasesteCheiDuplicateJson` inspectează textul brut deoarece `JSON.parse` ar păstra numai ultima valoare a unei chei duplicate. Dacă există erori, procesul se oprește cu `exit(1)`; altfel afișează confirmarea clară.

CUM ÎL DEMONSTREZI: Pornesc serverul și arăt mesajul de validare; apoi explic pe o copie ce eroare ar produce o imagine lipsă.

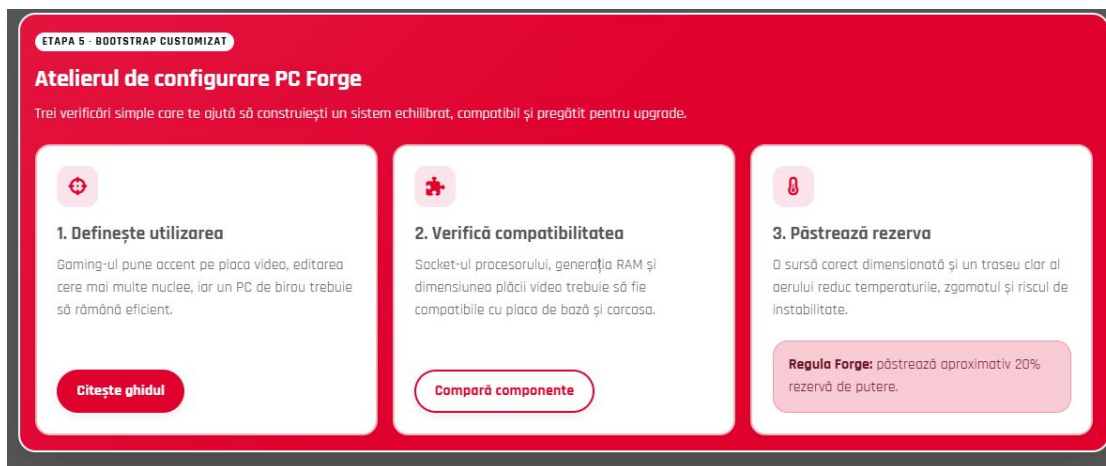
FIȘIERELE PE CARE LE ARĂȚI: `index.js`

BUCATA-CHEIE: `valideazaDateErori(caleErori); gasesteCheiDuplicateJson(continut)`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `index.js`. Ideea principală este: Funcția `valideazaDateErori` citește fișierul și acumulează toate problemele. `gasesteCheiDuplicateJson` inspectează textul brut deoarece `JSON.parse` ar păstra numai ultima valoare a unei chei duplicate. Dacă există erori, procesul se oprește cu `exit(1)`; altfel afișează confirmarea clară. În browser demonstrez astfel: Pornesc serverul și arăt mesajul de validare; apoi explic pe o copie ce eroare ar produce o imagine lipsă.

ETAPA 5 — CSS avansat, Sass și galerie

REZULTATUL AUDITULUI: Sunt documentate 5 bonusuri. Fiecare a fost urmărit de la cerință până la fișierul și demonstrația sa.



Reper vizual pentru Etapa 5

Bonus 5.1 — Galerie animată aleator

CE CERE: Numărul și secvența imaginilor se schimbă la încărcare, iar animația este generată în Sass.

CUM ESTE REZOLVAT ȘI DE CE: Serverul alege un număr divizibil cu 3 și un offset, EJS afișează imaginile consecutive, iar @for creează cadrele. Hover pune animation-play-state pe paused.

CUM ÎL DEMONSTREZI: Reîncarc de două ori, apoi țin mouse-ul peste galerie.

FIȘIERELE PE CARE LE ARĂȚI: index.js; views/fragmente/galerie-animata.ejs; resurse/sass/galerie-animata.scss

BUCATA-CHEIE: animation-play-state: paused

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în index.js. Ideea principală este: Serverul alege un număr divizibil cu 3 și un offset, EJS afișează imaginile consecutive, iar @for creează cadrele. Hover pune animation-play-state pe paused. În browser demonstrez astfel: Reîncarc de două ori, apoi țin mouse-ul peste galerie.

Bonus 5.2 — Efecte CSS suplimentare

CE CERE: Bonus deschis pentru efecte peste cerințele de bază.

CUM ESTE REZOLVAT ȘI DE CE: Proiectul folosește selecție personalizată, text în mișcare, separator stilizat, duotone cu pseudo-elemente și mix-blend-mode, plus reflexie portabilă cu duplicarea textului și blur.

CUM ÎL DEMONSTREZI: Arăt fiecare efect în pagina principală și descriu proprietatea esențială.

FIȘIERELE PE CARE LE ARĂȚI: resurse/css/stil.css; views/pagini/index.ejs

BUCATA-CHEIE: mix-blend-mode; filter: blur(...); @keyframes

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `resurse/css/stil.css`. Ideea principală este: Proiectul folosește selecție personalizată, text în mișcare, separator stilizat, duotone cu pseudo-elemente și `mix-blend-mode`, plus reflexie portabilă cu duplicarea textului și blur. În browser demonstrez astfel: Arăt fiecare efect în pagina principală și descriu proprietatea esențială.

Bonus 5.3 — Backup cu timestamp

CE CERE: CSS-ul anterior trebuie salvat înainte de recompilare.

CUM ESTE REZOLVAT ȘI DE CE: Numele copiei conține data și ora, deci versiunile nu se suprascriu și pot fi ordonate cronologic.

CUM ÎL DEMONSTREZI: Arăt folderul `backup/resurse/css` și două nume diferite.

FIȘIERELE PE CARE LE ARĂȚI: `index.js`

BUCATA-CHEIE: `new Date().toISOString()`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `index.js`. Ideea principală este: Numele copiei conține data și ora, deci versiunile nu se suprascriu și pot fi ordonate cronologic. În browser demonstrez astfel: Arăt folderul `backup/resurse/css` și două nume diferite.

Bonus 5.4 — Nume cu mai multe puncte

CE CERE: Compilatorul trebuie să accepte corect nume precum `tema.dark.scss`.

CUM ESTE REZOLVAT ȘI DE CE: `path.parse` separă `dirname`, `name` și `ext` fără a tăia greșit la primul punct; expresia regulată schimbă numai extensia finală `.scss`.

CUM ÎL DEMONSTREZI: Indic apelul pe un exemplu de nume și destinația CSS.

FIȘIERELE PE CARE LE ARĂȚI: `index.js`

BUCATA-CHEIE: `path.parse(caleFisier); /[.]scss$/i`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `index.js`. Ideea principală este: `path.parse` separă `dirname`, `name` și `ext` fără a tăia greșit la primul punct; expresia regulată schimbă numai extensia finală `.scss`. În browser demonstrez astfel: Indic apelul pe un exemplu de nume și destinația CSS.

Bonus 5.5 — Validarea galeriei JSON

CE CERE: Configurația și imaginile galeriei sunt verificate înainte de folosire.

CUM ESTE REZOLVAT ȘI DE CE: `valideazaGalerie` controlează structura, câmpurile, orele și fișierele. Astfel eroarea apare la pornire, nu ca imagine spartă în browser.

CUM ÎL DEMONSTREZI: Pornesc serverul și indic mesajul de succes.

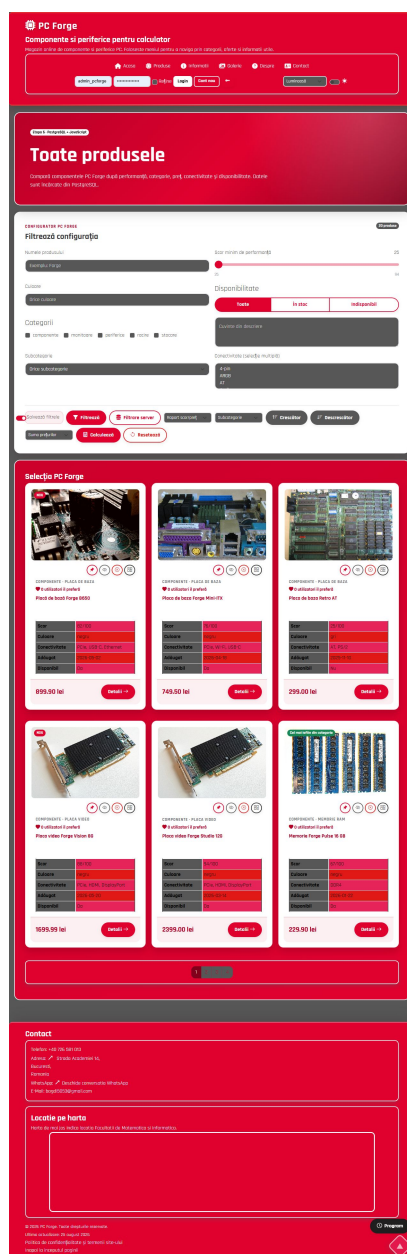
FIȘIERELE PE CARE LE ARĂȚI: `index.js`

BUCATA-CHEIE: `valideazaGalerie(...)`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în index.js. Ideea principală este: valideazaGalerie controlează structura, câmpurile, orele și fișierele. Astfel eroarea apare la pornire, nu ca imagine spartă în browser. În browser demonstrez astfel: Pornesc serverul și indic mesajul de succes.

ETAPA 6 — Produse, filtre și bonusuri de date

REZULTATUL AUDITULUI: Sunt documentate 20 bonusuri. Fiecare a fost urmărit de la cerință până la fișierul și demonstrația sa.



Reper vizual pentru Etapa 6

Bonus 6.1 — Inputuri generate din date

CE CERE: Opțiunile filtrelor trebuie să se potrivească produselor.

CUM ESTE REZOLVAT ȘI DE CE: Serverul extrage limitele și valorile distincte; EJS construiește range, datalist, checkbox, select simplu și multiplu. Astfel nu apar opțiuni inventate.

CUM ÎL DEMONSTREZI: Arăt filtrele și apoi valorile pregătite în rută.

FIȘIERELE PE CARE LE ARĂȚI: index.js; views/pagini/produse.ejs

BUCATA-CHEIE: Math.min(...); new Set(...)

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în index.js. Ideea principală este: Serverul extrage limitele și valorile distincte; EJS construiește range, datalist, checkbox, select simplu și multiplu. Astfel nu apar opțiuni inventate. În browser demonstrez astfel: Arăt filtrele și apoi valorile pregătite în rută.

Bonus 6.2 — Trei teme persistente

CE CERE: Vizitatorul poate selecta trei teme.

CUM ESTE REZOLVAT ȘI DE CE: tema.js aplică o clasă sau un atribut pe document și păstrează alegerea în localStorage; utilizatorul autentificat o poate păstra și în bază.

CUM ÎL DEMONSTREZI: Schimb tema, navighez și reîncarc.

FIȘIERELE PE CARE LE ARĂȚI: resurse/js/tema.js; resurse/css/teme.css

BUCATA-CHEIE: localStorage.setItem(...)

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în resurse/js/tema.js. Ideea principală este: tema.js aplică o clasă sau un atribut pe document și păstrează alegerea în localStorage; utilizatorul autentificat o poate păstra și în bază. În browser demonstrez astfel: Schimb tema, navighez și reîncarc.

Bonus 6.3 — Mesaj fără rezultate

CE CERE: Lista trebuie să explice atunci când niciun produs nu trece filtrele.

CUM ESTE REZOLVAT ȘI DE CE: După filtrare se numără cardurile vizibile, iar mesajul dedicat este afișat numai când numărul devine zero.

CUM ÎL DEMONSTREZI: Introduc un nume imposibil.

FIȘIERELE PE CARE LE ARĂȚI: resurse/js/produse.js

BUCATA-CHEIE: mesaj.style.display = vizibile === 0 ? ...

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în resurse/js/produse.js. Ideea principală este: După filtrare se numără cardurile vizibile, iar mesajul dedicat este afișat numai când numărul devine zero. În browser demonstrez astfel: Introduc un nume imposibil.

Bonus 6.4 — Filtrare imediată

CE CERE: Rezultatele se schimbă la input/change.

CUM ESTE REZOLVAT ȘI DE CE: Ascultătorii sunt atașați controalelor; algoritmul recitește valorile și actualizează clasele fără reload.

CUM ÎL DEMONSTREZI: Mut range-ul și bifez o categorie.

FIȘIERELE PE CARE LE ARĂȚI: resurse/js/produse.js

BUCATA-CHEIE: `addEventListener('input', ...); addEventListener('change', ...)`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `resurse/js/produse.js`. Ideea principală este: Ascultătorii sunt atașați controalelor; algoritmul recitește valorile și actualizează clasele fără reload. În browser demonstrez astfel: Mut range-ul și bifez o categorie.

Bonus 6.5 — Paginare calculată

CE CERE: Numărul paginilor depinde de rezultate.

CUM ESTE REZOLVAT ȘI DE CE: Numărul de pagini este $\text{Math.ceil}(N/K)$; schimbarea paginii arată numai intervalul corespunzător și reconstruiește controalele.

CUM ÎL DEMONSTREZI: Reduc rezultatele și urmăresc cum scade numărul paginilor.

FIȘIERELE PE CARE LE ARĂȚI: `resurse/js/produse.js`

BUCATA-CHEIE: `Math.ceil(produse.length / produsePePagina)`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `resurse/js/produse.js`. Ideea principală este: Numărul de pagini este $\text{Math.ceil}(N/K)$; schimbarea paginii arată numai intervalul corespunzător și reconstruiește controalele. În browser demonstrez astfel: Reduc rezultatele și urmăresc cum scade numărul paginilor.

Bonus 6.6 — Fixare și ascundere temporară

CE CERE: Un card poate fi fixat sau ascuns, inclusiv pe durata sesiunii.

CUM ESTE REZOLVAT ȘI DE CE: Clasele schimbă poziția/vizibilitatea; `sessionStorage` păstrează ascunderea numai în fila curentă, exact cât cere o stare temporară.

CUM ÎL DEMONSTREZI: Fixez un card, ascund altul și reîncarc.

FIȘIERELE PE CARE LE ARĂȚI: `resurse/js/produse.js`

BUCATA-CHEIE: `sessionStorage.setItem(...)`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `resurse/js/produse.js`. Ideea principală este: Clasele schimbă poziția/vizibilitatea; `sessionStorage` păstrează ascunderea numai în fila curentă, exact cât cere o stare temporară. În browser demonstrez astfel: Fixez un card, ascund altul și reîncarc.

Bonus 6.7 — Căutare fără diacritice

CE CERE: Căutarea tratează â/a și ș/s ca echivalente.

CUM ESTE REZOLVAT ȘI DE CE: `normalize('NFD')` descompune litera și diacriticul, iar expresia regulată elimină semnele combinante înainte de lowercase.

CUM ÎL DEMONSTREZI: Caut același text cu și fără diacritice.

FIȘIERELE PE CARE LE ARĂȚI: `resurse/js/produse.js`

BUCATA-CHEIE: `text.normalize('NFD').replace(/[\u0300-\u036f]/g, "")`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `resurse/js/produse.js`. Ideea principală este: `normalize('NFD')` descompune litera și diacriticul, iar expresia regulată elimină semnele combinante înainte de lowercase. În browser demonstrez astfel: Caut același text cu și fără diacritice.

Bonus 6.8 — Sortare după două chei

CE CERE: Egalitățile primei chei se rezolvă prin a doua.

CUM ESTE REZOLVAT ȘI DE CE: Comparatorul compară mai întâi raportul caracteristică/preț, apoi subcategoria. Dacă prima diferență este zero, întoarce comparația secundară.

CUM ÎL DEMONSTREZI: Sortez și explic două produse cu aceeași primă valoare.

FIȘIERELE PE CARE LE ARĂȚI: `resurse/js/produse.js`

BUCATA-CHEIE: `return diferentaPrincipala || diferentaSecundara`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `resurse/js/produse.js`. Ideea principală este: Comparatorul compară mai întâi raportul caracteristică/preț, apoi subcategoria. Dacă prima diferență este zero, întoarce comparația secundară. În browser demonstrez astfel: Sortez și explic două produse cu aceeași primă valoare.

Bonus 6.9 — Imagini derivate pentru produs

CE CERE: Caruselul trebuie să aibă minimum trei imagini ale produsului curent.

CUM ESTE REZOLVAT ȘI DE CE: `pregatesteVariante` folosește Sharp pentru a genera două versiuni WebP din fotografia acelui produs. `obtinImaginiProdus` întoarce originalul și derivatele, nu fotografiile ale altor produse.

CUM ÎL DEMONSTREZI: Deschid pagina unui produs și arăt cele trei cadre.

FIȘIERELE PE CARE LE ARĂȚI: `module/imagini-produse.js`; `index.js`

BUCATA-CHEIE: `sharp(sursa).resize(...).webp(...).toFile(...)`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `module/imagini-produse.js`. Ideea principală este: `pregatesteVariante` folosește Sharp pentru a genera două versiuni WebP din fotografia acelui produs. `obtinImaginiProdus` întoarce originalul și derivatele, nu fotografiile ale altor produse. În browser demonstrez astfel: Deschid pagina unui produs și arăt cele trei cadre.

Bonus 6.10 — Filtrare și sortare pe server

CE CERE: Cerința poate fi demonstrată și fără filtrare exclusiv în client.

CUM ESTE REZOLVAT ȘI DE CE: Fetch trimite criteriile, Express validează parametrii și întoarce numai produsele potrivite, ordonate. Clientul reconstruiește zona de rezultate.

CUM ÎL DEMONSTREZI: Aplic filtrul server și urmăresc cererea în Network.

FIȘIERELE PE CARE LE ARĂȚI: index.js; resurse/js/produse.js

BUCATA-CHEIE: `fetch('/api/produse?...')`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în index.js. Ideea principală este: Fetch trimite criteriile, Express validează parametrii și întoarce numai produsele potrivite, ordonate. Clientul reconstruiește zona de rezultate. În browser demonstrez astfel: Aplic filtrul server și urmăresc cererea în Network.

Bonus 6.11 — Modal Bootstrap

CE CERE: Detalii scurte apar într-un modal real Bootstrap.

CUM ESTE REZOLVAT ȘI DE CE: Atributele data-bs deschid componenta; evenimentul show actualizează conținutul pentru cardul apăsat.

CUM ÎL DEMONSTREZI: Apăs butonul de detalii și închid cu tastatura.

FIȘIERELE PE CARE LE ARĂȚI: views/pagini/produse.ejs; resurse/js/produse.js

BUCATA-CHEIE: `bootstrap.Modal; show.bs.modal`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în views/pagini/produse.ejs. Ideea principală este: Atributele data-bs deschid componenta; evenimentul show actualizează conținutul pentru cardul apăsat. În browser demonstrez astfel: Apăs butonul de detalii și închid cu tastatura.

Bonus 6.12 — Oferte temporizate

CE CERE: Reducerea depinde de timp și trebuie curățată după expirare.

CUM ESTE REZOLVAT ȘI DE CE: Serverul citește ofertele JSON, calculează intervalul activ, aplică procentul și păstrează istoric. Timerul actualizează vizual timpul rămas.

CUM ÎL DEMONSTREZI: Deschid zona de oferte și explic începutul, finalul și prețul redus.

FIȘIERELE PE CARE LE ARĂȚI: index.js; resurse/json/oferte.json

BUCATA-CHEIE: `pretRedus = pret * (1 - procent / 100)`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în index.js. Ideea principală este: Serverul citește ofertele JSON, calculează intervalul activ, aplică procentul și păstrează istoric. Timerul actualizează vizual timpul rămas. În browser demonstrez astfel: Deschid zona de oferte și explic începutul, finalul și prețul redus.

Bonus 6.13 — Curățarea backupurilor vechi

CE CERE: Fișierele CSS mai vechi de T minute trebuie șterse recursiv.

CUM ESTE REZOLVAT ȘI DE CE: `curataBackupuriCssVechi` citește pragul `minutePastrareBackupCss`, verifică extensia `.css` și `mtime`, apoi șterge numai în folderul `backup/resurse/css`. Rulează la pornire și periodic; `timer.unref` nu ține singur serverul pornit.

CUM ÎL DEMONSTREZI: Arăt opțiunea de 43200 minute și testul cu CSS vechi/nou.

FIȘIERELE PE CARE LE ARĂȚI: `module/curatare-backup.js`; `resurse/json/optiuni-server.json`

BUCATA-CHEIE: `if (vechimeMinute > minutePastrare) unlink(...)`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `module/curatare-backup.js`. Ideea principală este: `curataBackupuriCssVechi` citește pragul `minutePastrareBackupCss`, verifică extensia `.css` și `mtime`, apoi șterge numai în folderul `backup/resurse/css`. Rulează la pornire și periodic; `timer.unref` nu ține singur serverul pornit. În browser demonstrez astfel: Arăt opțiunea de 43200 minute și testul cu CSS vechi/nou.

Bonus 6.14 — Cel mai ieftin din categorie

CE CERE: Produsul minim din fiecare categorie este evidențiat.

CUM ESTE REZOLVAT ȘI DE CE: Un Map reține minimul curent pe categorie; o a doua trecere marchează produsele câștigătoare.

CUM ÎL DEMONSTREZI: Arăt insigna și două prețuri din aceeași categorie.

FIȘIERELE PE CARE LE ARĂȚI: `index.js`; `views/pagini/produse.ejs`

BUCATA-CHEIE: `minime.set(categorie, produs)`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `index.js`. Ideea principală este: Un Map reține minimul curent pe categorie; o a doua trecere marchează produsele câștigătoare. În browser demonstrez astfel: Arăt insigna și două prețuri din aceeași categorie.

Bonus 6.15 — Contor produse vizibile

CE CERE: Utilizatorul vede câte rezultate au rămas.

CUM ESTE REZOLVAT ȘI DE CE: Contorul se actualizează după fiecare filtrare și paginare, folosind aceeași listă calculată.

CUM ÎL DEMONSTREZI: Schimb un filtru și urmăresc numărul.

FIȘIERELE PE CARE LE ARĂȚI: `resurse/js/produse.js`

BUCATA-CHEIE: `contor.textContent = rezultate.length`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `resurse/js/produse.js`. Ideea principală este: Contorul se actualizează după fiecare filtrare și paginare, folosind aceeași listă calculată. În browser demonstrez astfel: Schimb un filtru și urmăresc numărul.

Bonus 6.16 — Produse similare

CE CERE: Pagina produsului recomandă produse relevante.

CUM ESTE REZOLVAT ȘI DE CE: Interogarea caută aceeași categorie și exclude id-ul curent, apoi limitează rezultatul.

CUM ÎL DEMONSTREZI: Deschid /produs/:id și compar categoriile.

FIȘIERELE PE CARE LE ARĂȚI: index.js; views/pagini/produs.ejs

BUCATA-CHEIE: WHERE categorie = \$1 AND id <> \$2

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în index.js. Ideea principală este: Interogarea caută aceeași categorie și exclude id-ul curent, apoi limitează rezultatul. În browser demonstrez astfel: Deschid /produs/:id și compar categoriile.

Bonus 6.17 — Reducere pentru seturi

CE CERE: Un set de produse primește o reducere calculată.

CUM ESTE REZOLVAT ȘI DE CE: Tabelele relaționale descriu setul și componentele; procentul este $\min(5, n) * 5$, astfel regula are plafon și depinde de numărul de produse.

CUM ÎL DEMONSTREZI: Arăt secțiunea de seturi și calculul.

FIȘIERELE PE CARE LE ARĂȚI: database/04-etapa6-bonusuri.sql; index.js

BUCATA-CHEIE: Math.min(5, n) * 5

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în database/04-etapa6-bonusuri.sql. Ideea principală este: Tabelele relaționale descriu setul și componentele; procentul este $\min(5, n) * 5$, astfel regula are plafon și depinde de numărul de produse. În browser demonstrez astfel: Arăt secțiunea de seturi și calculul.

Bonus 6.18 — Cele mai noi produse

CE CERE: Produsele recente sunt evidențiate și pot fi citite prin ORM.

CUM ESTE REZOLVAT ȘI DE CE: Sequelizee mapează tabela la un model și ordonează după data adăugării. EJS afișează marcajul Nou.

CUM ÎL DEMONSTREZI: Arăt secțiunea și modelul ORM.

FIȘIERELE PE CARE LE ARĂȚI: module/acces-orm.js; index.js

BUCATA-CHEIE: findAll({ order: [['data_adaugare', 'DESC']] })

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în module/acces-orm.js. Ideea principală este: Sequelizee mapează tabela la un model și ordonează după data adăugării. EJS afișează marcajul Nou. În browser demonstrez astfel: Arăt secțiunea și modelul ORM.

Bonus 6.19 — Program global

CE CERE: Programul magazinului se afișează consecvent.

CUM ESTE REZOLVAT ȘI DE CE: Un fragment EJS reutilizabil primește datele și evită duplicarea markupului în mai multe pagini.

CUM ÎL DEMONSTREZI: Arăt programul și include-ul fragmentului.

FIȘIERELE PE CARE LE ARĂȚI: views/fragmente/program.ejs

BUCATA-CHEIE: <%- include('fragmente/program', ...) %>

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în views/fragmente/program.ejs. Ideea principală este: Un fragment EJS reutilizabil primește datele și evită duplicarea markupului în mai multe pagini. În browser demonstrez astfel: Arăt programul și include-ul fragmentului.

Bonus 6.20 — Compararea produselor

CE CERE: Produsele alese persistă și sunt comparate separat.

CUM ESTE REZOLVAT ȘI DE CE: ID-urile sunt salvate în localStorage, limita este verificată, iar pagina /compara încarcă datele corespunzătoare.

CUM ÎL DEMONSTREZI: Selectez două produse, reîncarc și deschid comparația.

FIȘIERELE PE CARE LE ARĂȚI: resurse/js/comparare.js; views/pagini/compara.ejs

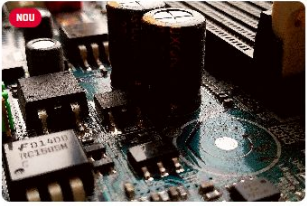
BUCATA-CHEIE: JSON.parse(localStorage.getItem(...))

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în resurse/js/comparare.js. Ideea principală este: ID-urile sunt salvate în localStorage, limita este verificată, iar pagina /compara încarcă datele corespunzătoare. În browser demonstrez astfel: Selectez două produse, reîncarc și deschid comparația.

ETAPA 7 — Bootstrap JS, arhitectură și persistență

REZULTATUL AUDITULUI: Sunt documentate 3 bonusuri. Fiecare a fost urmărit de la cerință până la fișierul și demonstrația sa.

Selecția PC Forge



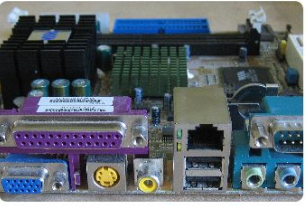
COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

Placă de bază Forge B650

Scor	82/100
Culoare	negru
Conectivitate	PCIe, USB-C, Ethernet
Adăugat	2026-05-02
Disponibil	Da

899.90 lei [Detalii →](#)



COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

Placă de bază Forge Mini-ITX

Scor	76/100
Culoare	negru
Conectivitate	PCIe, Wi-Fi, USB-C
Adăugat	2026-04-18
Disponibil	Da

749.50 lei [Detalii →](#)



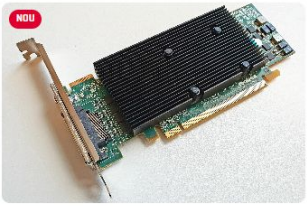
COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

Placă de bază Retro AT

Scor	25/100
Culoare	gri
Conectivitate	AT, PS/2
Adăugat	2025-11-10
Disponibil	Nu

299.00 lei [Detalii →](#)



Detalii →



Detalii →

Cel mai ieftin din categorie



Detalii →

Reper vizual pentru Etapa 7

Bonus 7.1 — Condiții SQL cu OR

CE CERE: Query builder-ul trebuie să combine și condiții alternative.

CUM ESTE REZOLVAT ȘI DE CE: Constructorul de condiții grupează expresiile OR în paranteze și păstrează valorile ca parametri \$1, \$2. Astfel precedența logică rămâne corectă și valorile nu sunt concatenate în SQL.

CUM ÎL DEMONSTREZI: Arăt un apel select cu grup OR și SQL-ul rezultat.

FIȘIERELE PE CARE LE ARĂȚI: module/acces-bd.js

BUCATA-CHEIE: (camp1=\$1 OR camp2=\$2)

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în module/acces-bd.js. Ideea principală este: Constructorul de condiții grupează expresiile OR în paranteze și păstrează valorile ca parametri \$1, \$2. Astfel precedența logică rămâne corectă și valorile nu sunt concatenate în SQL. În browser demonstrez astfel: Arăt un apel select cu grup OR și SQL-ul rezultat.

Bonus 7.2 — ORM și Singleton

CE CERE: Un ORM trebuie folosit într-o zonă, iar proiectul să aibă Singleton în altă parte.

CUM ESTE REZOLVAT ȘI DE CE: Sequelize modelează produsele și citește cele mai noi în stil obiectual. AccesBD.getInstanta reutilizează un singur manager al Pool-ului PostgreSQL; Singleton nu înseamnă o singură conexiune fizică.

CUM ÎL DEMONSTREZI: Arăt modelul Sequelize și apoi getInstanta.

FIȘIERELE PE CARE LE ARĂȚI: module/acces-orm.js; module/acces-bd.js

BUCATA-CHEIE: static getInstanta() { ... }

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în module/acces-orm.js. Ideea principală este: Sequelize modelează produsele și citește cele mai noi în stil obiectual. AccesBD.getInstanta reutilizează un singur manager al Pool-ului PostgreSQL; Singleton nu înseamnă o singură conexiune fizică. În browser demonstrez astfel: Arăt modelul Sequelize și apoi getInstanta.

Bonus 7.3 — Filtre persistente

CE CERE: Ultima configurație de filtrare se poate salva în browser.

CUM ESTE REZOLVAT ȘI DE CE: La bifarea opțiunii, valorile controalelor sunt serializate în localStorage. La încărcare sunt restaurate și filtrarea se execută din nou; resetarea cere confirmare și șterge cheia.

CUM ÎL DEMONSTREZI: Salvez filtrele, reîncarc și apoi le resetez.

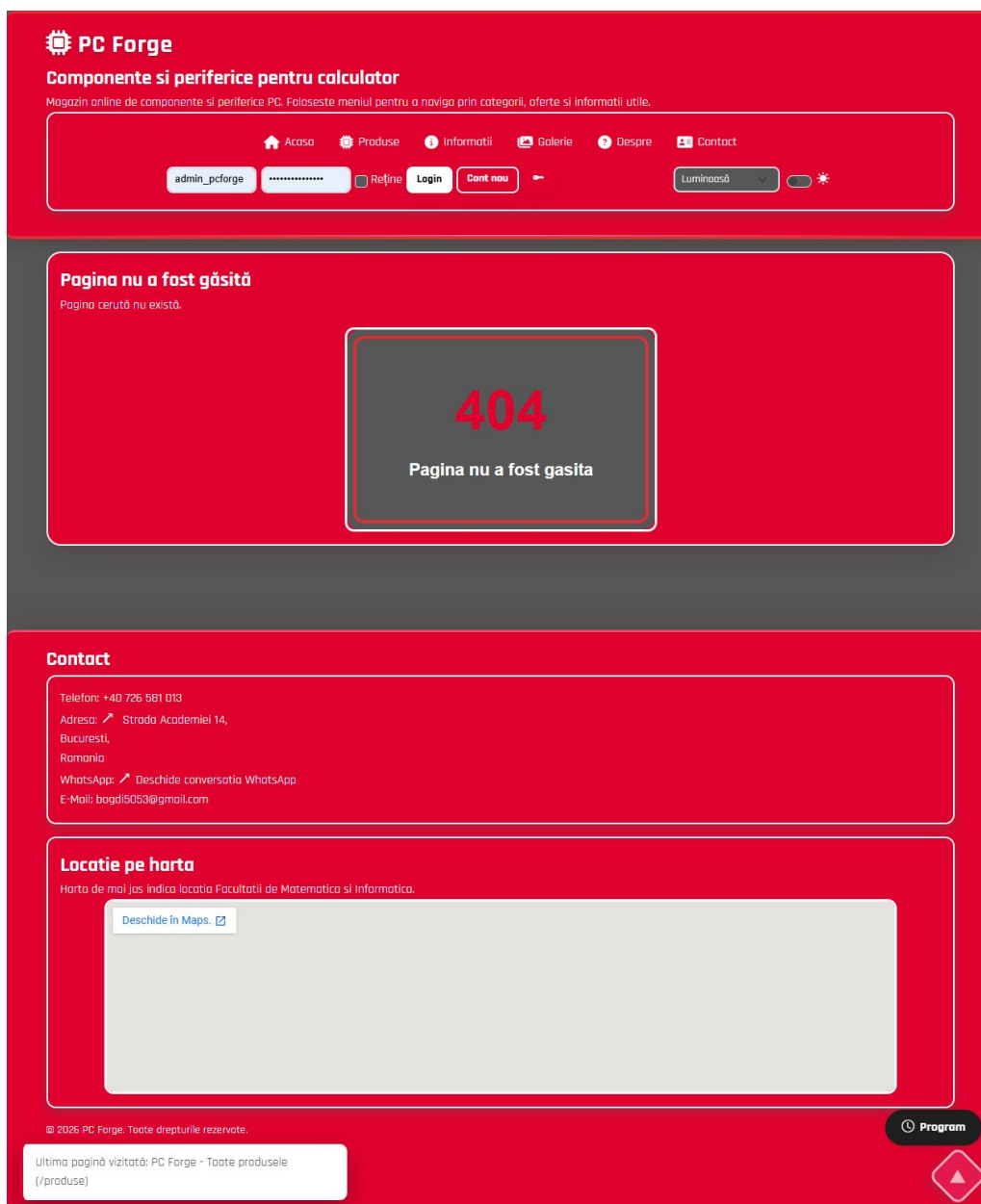
FIȘIERELE PE CARE LE ARĂȚI: resurse/js/produse.js

BUCATA-CHEIE: localStorage.setItem('filtre-produse', JSON.stringify(...))

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în resurse/js/produse.js. Ideea principală este: La bifarea opțiunii, valorile controalelor sunt serializate în localStorage. La încărcare sunt restaurate și filtrarea se execută din nou; resetarea cere confirmare și șterge cheia. În browser demonstrez astfel: Salvez filtrele, reîncarc și apoi le resetez.

ETAPA 8 — Utilizatori, securitate și administrare

REZULTATUL AUDITULUI: Sunt documentate 13 bonusuri. Fiecare a fost urmărit de la cerință până la fișierul și demonstrația sa.



Reper vizual pentru Etapa 8

Bonus 8.1 — Salt unic și script

CE CERE: Parolele trebuie stocate criptografic, cu salt diferit.

CUM ESTE REZOLVAT ȘI DE CE: randomBytes generează saltul, script derivă hashul, iar timingSafeEqual compară fără diferențe temporale ușor observabile. Parola nu este decriptată niciodată.

CUM ÎL DEMONSTREZI: Arăt funcțiile fără a expune parole sau .env.

FIȘIERELE PE CARE LE ARĂȚI: module/securitate.js

BUCATA-CHEIE: crypto.script(...); crypto.timingSafeEqual(...)

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în module/securitate.js. Ideea principală este: randomBytes generează saltul, script derivă hashul, iar timingSafeEqual compară fără diferențe temporale ușor observabile. Parola nu este decriptată niciodată. În browser demonstrez astfel: Arăt funcțiile fără a expune parole sau .env.

Bonus 8.2 — Tema salvată pe server

CE CERE: Preferința vizuală a utilizatorului autentificat persistă în cont.

CUM ESTE REZOLVAT ȘI DE CE: Endpointul validează tema și actualizează utilizatorul; la autentificare valoarea din bază completează interfața.

CUM ÎL DEMONSTREZI: Schimb tema în profil, ies și reintru.

FIȘIERELE PE CARE LE ARĂȚI: module/rute-utilizatori.js; resurse/js/tema.js

BUCATA-CHEIE: UPDATE utilizatori SET tema=\$1 WHERE id=\$2

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în module/rute-utilizatori.js. Ideea principală este: Endpointul validează tema și actualizează utilizatorul; la autentificare valoarea din bază completează interfața. În browser demonstrez astfel: Schimb tema în profil, ies și reintru.

Bonus 8.3 — Conturi neconfirmate

CE CERE: Utilizatorii neconfirmați primesc notificări și pot fi eliminați după termen.

CUM ESTE REZOLVAT ȘI DE CE: Jobul caută conturi vechi, trimite avertizarea și poate șterge conform configurației. Ștergerea este controlată de activeazaStergereConturiNeconfirmate, lăsată implicit false pentru a evita pierderi accidentale la dezvoltare.

CUM ÎL DEMONSTREZI: Arăt jobul și opțiunea; nu șterg conturi reale la prezentare.

FIȘIERELE PE CARE LE ARĂȚI: index.js; resurse/json/optiuni-server.json

BUCATA-CHEIE: activeazaStergereConturiNeconfirmate

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în index.js. Ideea principală este: Jobul caută conturi vechi, trimite avertizarea și poate șterge conform configurației. Ștergerea este controlată de activeazaStergereConturiNeconfirmate, lăsată implicit false pentru a evita pierderi accidentale la dezvoltare. În browser demonstrez astfel: Arăt jobul și opțiunea; nu șterg conturi reale la prezentare.

Bonus 8.4 — Resetarea parolei

CE CERE: Utilizatorul primește un link temporar pentru o parolă nouă.

CUM ESTE REZOLVAT ȘI DE CE: Un token aleator are termen de expirare și este invalidat după folosire. Serverul nu trimite parola veche și verifică din nou regulile parolei.

CUM ÎL DEMONSTREZI: Folosesc un cont de test și arăt expirarea în cod.

FIȘIERELE PE CARE LE ARĂȚI: module/rute-utilizatori.js; module/securitate.js

BUCATA-CHEIE: `crypto.randomBytes(...).toString('hex')`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în module/rute-utilizatori.js. Ideea principală este: Un token aleator are termen de expirare și este invalidat după folosire. Serverul nu trimite parola veche și verifică din nou regulile parolei. În browser demonstrez astfel: Folosesc un cont de test și arăt expirarea în cod.

Bonus 8.5 — Sugestii de username

CE CERE: La conflict se oferă variante libere.

CUM ESTE REZOLVAT ȘI DE CE: Serverul generează candidați din nume și sufixe, îi verifică în bază și întoarce numai variante neocupate.

CUM ÎL DEMONSTREZI: Încerc un username existent și arăt sugestiile.

FIȘIERELE PE CARE LE ARĂȚI: module/rute-utilizatori.js

BUCATA-CHEIE: `genereazaSugestiiUsername(...)`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în module/rute-utilizatori.js. Ideea principală este: Serverul generează candidați din nume și sufixe, îi verifică în bază și întoarce numai variante neocupate. În browser demonstrez astfel: Încerc un username existent și arăt sugestiile.

Bonus 8.6 — Blocare după loginuri eșuate

CE CERE: Încercările repetate trebuie limitate.

CUM ESTE REZOLVAT ȘI DE CE: Contorul și momentul blocării sunt actualizate la eșec; înainte de verificarea parolei serverul controlează dacă intervalul de blocare a expirat.

CUM ÎL DEMONSTREZI: Demonstrez numai pe un cont creat pentru test.

FIȘIERELE PE CARE LE ARĂȚI: module/rute-utilizatori.js

BUCATA-CHEIE: `incercari_esuate; blocat_pana`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în module/rute-utilizatori.js. Ideea principală este: Contorul și momentul blocării sunt actualizate la eșec; înainte de verificarea parolei serverul controlează dacă intervalul de blocare a expirat. În browser demonstrez astfel: Demonstrez numai pe un cont creat pentru test.

Bonus 8.7 — Mod mentenanță

CE CERE: În perioada configurată, utilizatorii obișnuiți văd 503, iar administratorul poate intra.

CUM ESTE REZOLVAT ȘI DE CE: Middleware-ul rulează înaintea rutelor, compară timpul curent cu intervalul și verifică rolul. 503 descrie indisponibilitate temporară.

CUM ÎL DEMONSTREZI: Arăt configurația și pagina de mentenanță.

FIȘIERELE PE CARE LE ARĂȚI: index.js

BUCATA-CHEIE: res.status(503).render(...)

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în index.js. Ideea principală este: Middleware-ul rulează înaintea rutelor, compară timpul curent cu intervalul și verifică rolul. 503 descrie indisponibilitate temporară. În browser demonstrez astfel: Arăt configurația și pagina de mentenanță.

Bonus 8.8 — Timp de la ultima autentificare

CE CERE: Profilul afișează o formulare umană.

CUM ESTE REZOLVAT ȘI DE CE: Diferența dintre Date.now și data ultimei autentificări este împărțită în secunde, minute, ore sau zile.

CUM ÎL DEMONSTREZI: Deschid profilul și indic valoarea calculată.

FIȘIERELE PE CARE LE ARĂȚI: module/rute-utilizatori.js; views/pagini/profil.ejs

BUCATA-CHEIE: Date.now() - new Date(data).getTime()

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în module/rute-utilizatori.js. Ideea principală este: Diferența dintre Date.now și data ultimei autentificări este împărțită în secunde, minute, ore sau zile. În browser demonstrez astfel: Deschid profilul și indic valoarea calculată.

Bonus 8.9 — Rămâi conectat persistent

CE CERE: Preferința nu este doar un checkbox al formularului, ci aparține contului.

CUM ESTE REZOLVAT ȘI DE CE: Coloana ramai_conectat este editabilă în profil. La login, serverul combină preferința salvată cu opțiunea curentă și extinde cookie-ul sesiunii la 30 de zile, fără a salva parola.

CUM ÎL DEMONSTREZI: Activez opțiunea în profil, ies, reintru și arăt maxAge.

FIȘIERELE PE CARE LE ARĂȚI: database/06-bonusuri-etapele6-8.sql; module/rute-utilizatori.js; views/pagini/profil.ejs

BUCATA-CHEIE: req.session.cookie.maxAge = 30 * 24 * 60 * 60 * 1000

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în database/06-bonusuri-etapele6-8.sql. Ideea principală este: Coloana ramai_conectat este editabilă în profil. La login, serverul combină preferința salvată cu opțiunea curentă și extinde cookie-ul sesiunii la 30 de zile, fără a salva parola. În browser demonstrez astfel: Activez opțiunea în profil, ies, reintru și arăt maxAge.

Bonus 8.10 — E-mail după inactivitate

CE CERE: Utilizatorii inactivi primesc o ofertă personalizată cu imagine.

CUM ESTE REZOLVAT ȘI DE CE: Jobul selectează conturile eligibile, atașează imaginea cu Content-ID și evită retrimiteria necontrolată prin data ultimei notificări.

CUM ÎL DEMONSTREZI: Arăt template-ul și atașamentul; nu trimit în masă la prezentare.

FIȘIERELE PE CARE LE ARĂȚI: index.js; module/email.js

BUCATA-CHEIE: attachments: [{ filename, path, cid }]

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în index.js. Ideea principală este: Jobul selectează conturile eligibile, atașează imaginea cu Content-ID și evită retrimiteria necontrolată prin data ultimei notificări. În browser demonstrez astfel: Arăt template-ul și atașamentul; nu trimit în masă la prezentare.

Bonus 8.11 — Drepturi active din baza de date

CE CERE: Drepturile depind de rolurile active și intervalele lor.

CUM ESTE REZOLVAT ȘI DE CE: areDreptul acceptă Symbol pentru modelul local și string/id pentru verificarea asincronă în tabelele utilizatori_roluri, roluri_drepturi și drepturi. Interogarea verifică data de început și sfârșit; pagina admin poate modifica intervalul atribuirii.

CUM ÎL DEMONSTREZI: Atribui un rol temporar unui cont de test și compar accesul înainte/după expirare.

FIȘIERELE PE CARE LE ARĂȚI: module/utilizator.js; module/rute-utilizatori.js; views/pagini/administrare-roluri.ejs

BUCATA-CHEIE: async areDreptul(drept) { SELECT EXISTS (...) }

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în module/utilizator.js. Ideea principală este: areDreptul acceptă Symbol pentru modelul local și string/id pentru verificarea asincronă în tabelele utilizatori_roluri, roluri_drepturi și drepturi. Interogarea verifică data de început și sfârșit; pagina admin poate modifica intervalul atribuirii. În browser demonstrez astfel: Atribui un rol temporar unui cont de test și compar accesul înainte/după expirare.

Bonus 8.12 — Maximum EJS și notificare

CE CERE: Datele și deciziile de afișare trebuie mutate cât mai mult în template.

CUM ESTE REZOLVAT ȘI DE CE: Ruta transmite obiectele, EJS decide markupul și clasele, iar după modificarea produsului un mesaj temporar este creat și dispare după trei secunde.

CUM ÎL DEMONSTREZI: Modific un produs de test și cronometrez mesajul.

FIȘIERELE PE CARE LE ARĂȚI: views/pagini/administrare-produse.ejs; resurse/js/admin-produse.js

BUCATA-CHEIE: setTimeout(() => mesaj.remove(), 3000)

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `views/pagini/administrare-produse.ejs`. Ideea principală este: Ruta transmite obiectele, EJS decide markupul și clasele, iar după modificarea produsului un mesaj temporar este creat și dispare după trei secunde. În browser demonstrez astfel: Modific un produs de test și cronometrez mesajul.

Bonus 8.13 — Favorite și stoc redus

CE CERE: Favoritele au listă, număr, top, reumplere și alertă de stoc cu imagine și link spre coș.

CUM ESTE REZOLVAT ȘI DE CE: Relația many-to-many leagă utilizatorul de produs. Endpointul actualizează favoritele, agregarea produce topul, iar jobul de stoc atașează imaginea cu CID și un URL absolut. Ruta `/cos/adauga/:id` pune produsul într-un coș minim de sesiune, suficient ca linkul e-mailului să aibă destinație funcțională.

CUM ÎL DEMONSTREZI: Adaug un favorit, deschid `/favorite` și `/top-favorite`, apoi arăt fluxul de stoc pe un produs test.

FIȘIERELE PE CARE LE ARĂȚI: `module/rute-utilizatori.js`; `views/pagini/favorite.ejs`; `views/pagini/cos.ejs`

BUCATA-CHEIE: `req.session.cos = [...new Set([...cos, idProdus])]`

FORMULARE PENTRU PROFESOARĂ: Acest bonus este implementat în `module/rute-utilizatori.js`. Ideea principală este: Relația many-to-many leagă utilizatorul de produs. Endpointul actualizează favoritele, agregarea produce topul, iar jobul de stoc atașează imaginea cu CID și un URL absolut. Ruta `/cos/adauga/:id` pune produsul într-un coș minim de sesiune, suficient ca linkul e-mailului să aibă destinație funcțională. În browser demonstrez astfel: Adaug un favorit, deschid `/favorite` și `/top-favorite`, apoi arăt fluxul de stoc pe un produs test.

Ce nu trebuie revendicat ca bonus al Etapei 8

DELIMITARE IMPORTANTĂ: După lista de bonusuri a Etapei 8, PDF-ul introduce separat Coș virtual și spune că cerințele urmau să fie publicate. Subpunctele despre starea comenzilor, graficul D3, reducerile programate, vouchere și readucerea stocului depind de acel sistem viitor și nu sunt prezentate aici ca bonusuri finalizate ale Etapei 8.

CE EXISTĂ TOTUȘI: Ruta minimă /cos/adauga/:id și pagina /cos există pentru ca linkul din alerta de stoc redus să fie funcțional. Ea nu este descrisă drept un sistem complet de comandă sau checkout.

Schema de răspuns la orice întrebare

1. **CERINȚA:** Spun într-o propoziție ce comportament trebuia obținut.
2. **INTRAREA:** Numesc datele primite: formular, JSON, sesiune, cookie sau PostgreSQL.
3. **FUNCȚIA:** Explic pașii importanți, nu fiecare semn de punctuație.
4. **REZULTATUL:** Arăt efectul în browser ori mesajul din terminal.
5. **MOTIVUL:** Leg soluția de securitate, reutilizare, performanță sau claritate.